



IE3010

Network Programming

3rd Year, Semester II

NetMessenger: A Multi-Client Chat and File-Sharing Platform over TCP/IP

Submitted to
Sri Lanka Institute of Information Technology

In partial fulfillment of the requirements for the
Bachelor of Science Special Honors Degree in Information Technology

10/07/2026

Declaration

I certify that this report does not incorporate without acknowledgement, any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief it does not contain any material previously published or written by another person, except where due reference is made in text.

Registration Number: IT23695634

Name: Jayashanka K A A

Contents

1. Introduction	4
2. Personalisation and Submission Naming	5
3. System Architecture	6
4. Implementation Details	7
5. Protocol Commands	8
6. TCP Framing and File Transfer	10
7. Concurrency and Synchronisation	11
8. Error Handling and Disconnects	12
9. Server Logging and Storage	13
10. Testing and Results	14
11. GitHub and Development Process	15
12. Design Decisions and Assumptions	16
13. Conclusion	17

1. Introduction

This report documents the implementation of NetMessenger, a TCP client/server messaging and file-sharing application developed for the IE3010 Network Programming assessment. The system was implemented in C on CentOS Stream 10 using POSIX/BSD sockets and POSIX threads. The server is designed to support multiple simultaneous clients and provides user registration, user listing, broadcast messaging, private messaging, chat rooms, file transfer, logging and graceful disconnect handling.

The implementation uses a line-oriented protocol for control and messaging commands. File transfer is length-framed: the sender transmits a SENDFILE header containing the destination, filename and exact byte count, followed immediately by the raw file bytes. The server and client both maintain receive buffers so that TCP packetisation does not become an application-level framing dependency.

2. Personalisation and Submission Naming

The implementation was personalised using the registration number IT23695634. The numeric portion is 23695634; the last four digits are 5634, producing the server port 11634. The NID is 6956. The personalised values are embedded consistently in the source, log name and storage path.

Requirement	Implemented value
Registration number	IT23695634
Numeric portion	23695634
Last four digits	5634
TCP server port	11634
NID	NID:6956
Server source	server_5634.c
Client source	client_5634.c
Makefile	Makefile_5634
Server log	netmsg_IT23695634.log
Storage root	storage/IT23695634
Client download directory	downloads_IT23695634

3. System Architecture

The server creates a listening TCP socket and accepts incoming connections. Each accepted client is assigned to a client structure and serviced by a detached pthread. The client program uses one main thread for interactive commands and a receiver thread for asynchronous incoming messages and files.

Server lifecycle -:

- 1) Create a TCP socket using `socket(AF_INET, SOCK_STREAM, 0)`.
- 2) Enable `SO_REUSEADDR` so the server can be restarted without waiting for the old socket to expire.
- 3) Bind the socket to all local interfaces on port 11634.
- 4) Listen for incoming TCP connections.
- 5) Accept a client connection and allocate an available client slot.
- 6) Create a detached pthread to manage the connected client.
- 7) Require REGISTER as the first application command.
- 8) Process subsequent commands until QUIT, connection loss, or a framing error.

4. Implementation Details

Data structures -:

The implementation uses fixed-size arrays, which keeps the design simple and predictable for the assessment prototype.

Structure	Purpose
Client	Stores socket descriptor, registration state, username, receive buffer, pthread handle and per-client send mutex.
Room	Stores room name and an array of pointers to member Client structures.
clients[MAX_CLIENTS]	Shared table of up to 32 connected client slots.
rooms[MAX_ROOMS]	Shared table of up to 32 rooms.
state_mu	Protects shared client/room state.
log_mu	Protects concurrent writes to the server log.

Registration and username management -:

The first command from a client must be REGISTER <username>. Usernames are validated using alphanumeric characters plus underscore and hyphen, and the server rejects duplicate usernames. The client also applies a maximum 31-character before sending the command. A successful registration returns an OK response carrying the personalised NID.

```
anuradha@localhost:~/IE3010_IT23695634$ grep -E '#define (PORT|NID_TAG|REGNO|LOG_FILE|STORAGE_ROOT)' server_5634.c
#define PORT 11634
#define NID_TAG "NID:6956"
#define REGNO "IT23695634"
#define LOG_FILE "netmsg_IT23695634.log"
#define STORAGE_ROOT "storage/IT23695634"
anuradha@localhost:~/IE3010_IT23695634$
```

Figure 1: personalised server startup

5. Protocol Commands

The client interface uses slash commands locally and translates them to the required network commands. The server treats the resulting TCP lines as protocol messages.

Feature	Client command	Wire command / behaviour	Expected result
Registration	/register alice	REGISTER alice	OK REGISTERED alice NID:6956
List users	/list	LIST	USERS ... NID:6956
Broadcast	/bcast hello	BCAST hello	Other clients receive MSG BCAST ...
Private message	/pmsg bob hello	PMSG bob hello	Only Bob receives MSG PRIV ...
Join room	/join lab	JOIN lab	OK JOINED lab NID:6956
Leave room	/leave lab	LEAVE lab	OK LEFT lab NID:6956
List rooms	/rooms	ROOMS	ROOMS ... NID:6956
Room message	/rmsg lab hello	RMSG lab hello	Room members receive MSG ROOM ...
File transfer	/sendfile bob file.txt	SENDFILE bob file.txt <size>	Exact file bytes are stored and forwarded
Quit	/quit	QUIT	OK BYE NID:6956 and disconnect

```

=====
NetMessenger Client - IT23695634
=====
Commands:
/register <username>      - Register with the server
/list                    - List connected users
/bcast <message>         - Broadcast to all users
/pmsg <user> <message>   - Send private message
/join <room>             - Join or create a room
/leave <room>            - Leave a room
/rooms                  - List rooms
/rmsg <room> <message>   - Send message to a room
/sendfile <user|room> <path> - Send a file to a user or room
/quit                   - Disconnect and exit
=====
> /register alice
>
<< OK REGISTERED alice NID:6956
>
<< MSG PRESENCE alice JOINED
>
<< MSG PRESENCE bob JOINED
>
<< MSG PRESENCE dave JOINED
>
<< MSG PRESENCE erin JOINED

```

Figure 2: Five simultaneous clients

```

> /list
>
<< USERS alice,bob,dave,erin NID:6956
> █

```

Figure 3: LIST command

```

<< OK SENT NID:6956
> /pmsg bob This is a private message
>
<< OK SENT NID:6956

```

Figure 4: Private messaging

```

> /join networking
>
<< OK JOINED networking NID:6956
>
<< MSG ROOM networking alice Hello networking room
> █

```

Figure 5: Room messaging

```

<< MSG PRESENCE Carol JOINED
> /sendfile bob test.txt
[+] File sent: test.txt -> bob (32 bytes)
>
<< OK FILE_RECEIVED test.txt NID:6956
>
<< MSG PRESENCE bob LEFT
> /list
>
<< USERS alice,dave,erin,Carol NID:6956
> █

```

Figure 6: Test Quit

6. TCP Framing and File Transfer

Why buffering is required -:

TCP provides an ordered byte stream rather than message boundaries. A single `recv()` may return a partial command, multiple commands, a complete command plus part of the next message, or a file header together with the beginning of the file. The implementation therefore maintains a receive buffer per client.

Server line framing -:

The server `recv_line()` function searches the buffered bytes for a newline. When a complete line exists, it copies the command out and shifts any remaining bytes to the start of the buffer. This allows partial and multiple commands to be handled correctly without assuming a particular TCP segmentation pattern.

```
recv_line()
|
+--- search c->rxbuf for '\n'
|
+--- extract one complete line
|
+--- preserve remaining bytes
|
+--- process command
```

Exact-byte file transfer -:

The file protocol uses the header format `SENDFILE <target> <filename> <filesize>` followed immediately by the raw file bytes. On the receiving side, `recv_exact()` first consumes any bytes already buffered in the client structure and then continues receiving until exactly `filesize` bytes have been consumed. This prevents file data from being mistaken for a later command and also handles cases where the header and file data arrive in the same TCP read.

Server-side file path -: `storage/IT23695634/<sender_username>/<filename>`

Client-side download path -: `downloads_IT23695634/<filename>`

```
> /sendfile bob test.txt
[+] File sent: test.txt -> bob (32 bytes)
>
<< OK FILE_RECEIVED test.txt NID:6956
```

Figure 7: File transfer result

```
>
[+] File received from alice: test.txt (32 bytes) -> ./downloads_IT23695634/test.txt
>
```

Figure 8: File integrity verification

```

anuradha@localhost:~/IE3010_IT23695634$ cd ~/IE3010_IT23695634
ls -la storage/IT23695634/
total 0
drwxr-xr-x. 4 anuradha anuradha 35 Oct  4 08:21 .
drwxr-xr-x. 3 anuradha anuradha 24 Oct  4 03:45 ..
drwxr-xr-x. 2 anuradha anuradha 22 Oct  4 08:21 alice
drwxr-xr-x. 2 anuradha anuradha 60 Oct  4 07:43 anuradha
anuradha@localhost:~/IE3010_IT23695634$

```

Figure 9: Server file storage

7. Concurrency and Synchronisation

The server follows a thread-per-client model. Each accepted TCP connection is handled by a detached pthread. This enables simultaneous interaction between at least five clients while keeping the implementation understandable.

Shared resource	Protection	Reason
clients[] and rooms[]	state_mu	Prevents concurrent registration, room creation, membership updates and lookups from racing.
One client socket output	Client.send_mu	Prevents concurrent server threads from interleaving bytes on the same TCP connection.
Log file	log_mu	Prevents two client threads from mixing log lines during concurrent events.

Concurrency model -:

accept ()

pthread: client Alice

pthread: client Bob

pthread: client Carol

pthread: client Dave

pthread: client Erin

8. Error Handling and Disconnects

The implementation includes protocol validation and defensive handling for malformed commands, invalid usernames/room names, unknown users, unknown rooms, oversized files, storage errors and TCP framing errors. SIGPIPE is ignored so that attempting to write to a client that has already disconnected does not terminate the server process.

Situation	Server response / action
Duplicate username	ERR 001 USERNAME_TAKEN
Unknown user	ERR 002 USER_NOT_FOUND
Unknown room	ERR 003 ROOM_NOT_FOUND
File over 10 MiB	ERR 004 FILE_TOO_LARGE
Malformed / unknown command	ERR 005 ...
Line too long	ERR 005 LINE_TOO_LONG
Storage failure	ERR 007 STORAGE_ERROR
Client closes connection	Remove from rooms/users, log disconnect, broadcast LEFT presence

```
> /register hello
>
<< ERR 005 UNKNOWN_COMMAND NID:6956
> █
```

Figure 10: Error handling

9. Server Logging and Storage

The server writes timestamped events to netmsg_IT23695634.log. Logging covers startup, registration, broadcast/private/room messaging, file transfers, QUIT and disconnect events. The log mutex serialises file access when several client threads generate events at the same time.

```
[timestamp] SERVER START: Port 11634
[timestamp] REGISTER: User 'alice' registered.
[timestamp] SENDFILE: alice -> bob: 'test.txt' (... bytes)
```

```
[2026-10-04 07:37:31] RMSG: anuradha to room 'dev': 'Hello dev team!'
[2026-10-04 07:38:59] LEAVE: bob left room 'dev'
[2026-10-04 07:39:28] RMSG: anuradha to room 'dev': 'Can bob still see this?'
[2026-10-04 07:41:19] SENDFILE: anuradha -> bob: 'myfile.txt' (39 bytes)
[2026-10-04 07:42:45] JOIN: anuradha joined room 'testroom'
[2026-10-04 07:42:48] JOIN: bob joined room 'testroom'
[2026-10-04 07:43:36] SENDFILE: anuradha -> testroom: 'roomfile.txt' (23 bytes)
[2026-10-04 07:44:48] PMSG: anuradha -> nonexistentuser: 'Hello'
[2026-10-04 07:44:59] RMSG: anuradha to room 'fakeroom': 'Hello'
[2026-10-04 07:46:33] QUIT: bob
[2026-10-04 07:46:33] DISCONNECT: User 'bob' disconnected.
[2026-10-04 08:02:13] SERVER START: Port 11634
[2026-10-04 08:03:13] REGISTER: User 'alice' registered.
[2026-10-04 08:03:46] REGISTER: User 'bob' registered.
[2026-10-04 08:04:15] REGISTER: User 'dave' registered.
[2026-10-04 08:04:44] REGISTER: User 'erin' registered.
[2026-10-04 08:09:16] BCAST: alice -> 'Hello everyone'
[2026-10-04 08:10:31] PMSG: alice -> bob: 'This is a private message'
[2026-10-04 08:14:14] JOIN: alice joined room 'networking'
[2026-10-04 08:14:23] JOIN: bob joined room 'networking'
[2026-10-04 08:14:26] JOIN: dave joined room 'networking'
[2026-10-04 08:14:31] JOIN: erin joined room 'networking'
[2026-10-04 08:14:50] RMSG: alice to room 'networking': 'Hello networking room'
[2026-10-04 08:17:25] REGISTER: User 'Carol' registered.
[2026-10-04 08:18:40] LEAVE: Carol left room 'networking'
[2026-10-04 08:21:29] SENDFILE: alice -> bob: 'test.txt' (32 bytes)
anuradha@localhost:~/IE3010_IT23695634$
```

Figure 11: Server log evidence

10. Testing and Results

Testing was performed on the CentOS Stream 10 GUI environment after the final server and client code were compiled successfully. The implemented functional tests were executed and reported as passed during the development session.

Test ID	Test	Expected behaviour	Result
T01	Server startup	Server binds to TCP 11634 and displays NID:6956	PASS
T02	Five clients	At least five clients remain connected simultaneously	PASS
T03	Registration	Valid unique username receives registration confirmation	PASS
T04	Duplicate username	Second registration with same name is rejected	PASS
T05	LIST	Connected users are listed	PASS
T06	BCAST	All other registered clients receive the broadcast	PASS
T07	PMSG	Only the requested user receives the private message	PASS
T08	JOIN/LEAVE	Client joins and leaves a room successfully	PASS
T09	ROOMS	Available rooms are listed	PASS
T10	RMSG	Room members receive the room message	PASS
T11	SENDFILE user	File is stored and forwarded to target user	PASS
T12	SENDFILE room	File is forwarded to room members	PASS
T13	File integrity	Received file matches the original	PASS
T14	Invalid command	Server returns an error and continues	PASS
T15	Unknown user/room	Appropriate error code returned	PASS
T16	QUIT	Client disconnects cleanly and state is cleaned	PASS
T17	Unexpected disconnect	Client is removed and presence state updated	PASS

11. GitHub and Development Process

The project is maintained in a Git repository and the final repository should contain the personalised C source files, Makefile and documentation. The assessment brief requires at least eight meaningful incremental commits. Commit history should therefore reflect genuine development stages rather than artificially generated history.

12. Design Decisions and Assumptions

Decision / assumption	Rationale
Thread per client	Simple concurrency model that directly supports multiple simultaneous clients.
Global state mutex	Protects shared client and room data from races.
Per-client send mutex	Prevents concurrent writes to the same TCP connection from interleaving.
Buffered line framing	Handles partial lines and multiple lines in one <code>recv()</code> .
Exact file byte counting	Separates binary file content from subsequent text commands.
Maximum file size 10 MiB	Prevents unbounded storage use for this prototype.
Filename validation	Prevents path separator characters from escaping the intended storage directory.
Server-side receiver framing	The server stores and forwards file data using the announced byte count.
Client-side MSG FILE frame	Receiver-side delivery uses MSG FILE <sender> <filename> <size> followed by exact raw bytes.
Rooms remain allocated	The supplied command set has JOIN/LEAVE/ROOMS but no DELETE ROOM command; unused rooms therefore remain available.

The client/server pair was kept intentionally text-oriented for control commands while treating file payloads as opaque binary data. This separation simplifies debugging and makes the protocol behaviour visible during demonstrations.

13. Conclusion

The final NetMessenger implementation provides a multi-client TCP messaging service in C with personalised configuration for registration number IT23695634. The server supports concurrent clients using pthreads, shared state protected by mutexes, user and room management, broadcast and private messaging, file transfer, logging and disconnect cleanup. The client includes an independent receiver thread and buffered file handling so that asynchronous messages and binary file payloads can co-exist safely on the TCP connection.

The system was compiled and functionally tested on CentOS Stream 10. The remaining task before submission is to replace the screenshot placeholders in this report with the actual evidence captured during testing, confirm the GitHub repository and commit requirements, and create the final ZIP using the naming convention specified by the assessment brief.